ГУАП

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

канд. техн. наук		С. А. Чернышев
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ 3

НАВИГАЦИЯ

по курсу: Распределенные информационные системы

РАБОТУ ВЫПОЛНИЛ

Студент гр. № 4513МКз		А. А. Ледневская
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Репозиторий пректа в GitHub: https://github.com/anastasia-l-code/rate_flow

В ходе разработки проекта были выполнены следующие доработки:

1. «Перевести навигацию приложения на использование именованных маршрутов»

Создан файл app_routes_name.dart с константами имён маршрутов и зарегистрированы экраны в MaterialApp. Переходы между экранами переведен на Navigator.pushNamed(), а возврат реализовали через Navigator.pop().

lib > app > app_routes_name.dart > ...

```dart
abstract final class AppRoutesName {
  static const String main = '/';
  static const String currency = '/currency';
  static const String settings = '/settings';
  static const String sendText = '/send-text';
}
```

Рисунок 1 – файл app_routes_name.dart

```dart
initialRoute: AppRoutesName.main,
routes: {
  AppRoutesName.main: (_) => const HomePage(),
  AppRoutesName.settings: (_) => const SettingsPage(),
  AppRoutesName.currency: (context) {
    final title =
        ModalRoute.of(context)!.settings.arguments as String? ?? 'Валюта';
    return CurrencyDetailPage(title: title);
  },
},
```

Рисунок 2 – регистрация маршрутов в main.dart

2. «Подключить пакет flutter_native_splash и настроить нативный Splash Page, указав произвольный цвет фона и логотип приложения;»

Заставка настроена в pubspec.yaml для Android, iOS и Web



Рисунок 3 – файл pubspec.yaml и вид splash page

3. **«Добавить в маршрутную схему не менее двух дополнительных экранов:**

- один из экранов должен возвращать результат выполнения - Navigator.pop(context, result);

- на экране, откуда происходит переход, необходимо обработать возвращаемое значение (например, отобразить его в SnackBar, текстом или другим произвольным способом на UI).»

В проект добавлены экран настроек SettingsPage и экран ввода текста SendTextPage.

На экране ввода текста реализовано поле и кнопка «Отправить», которая закрывает экран через Navigator.pop(context, text). На экране настроек результат получаем через await и отображаем в SnackBar.

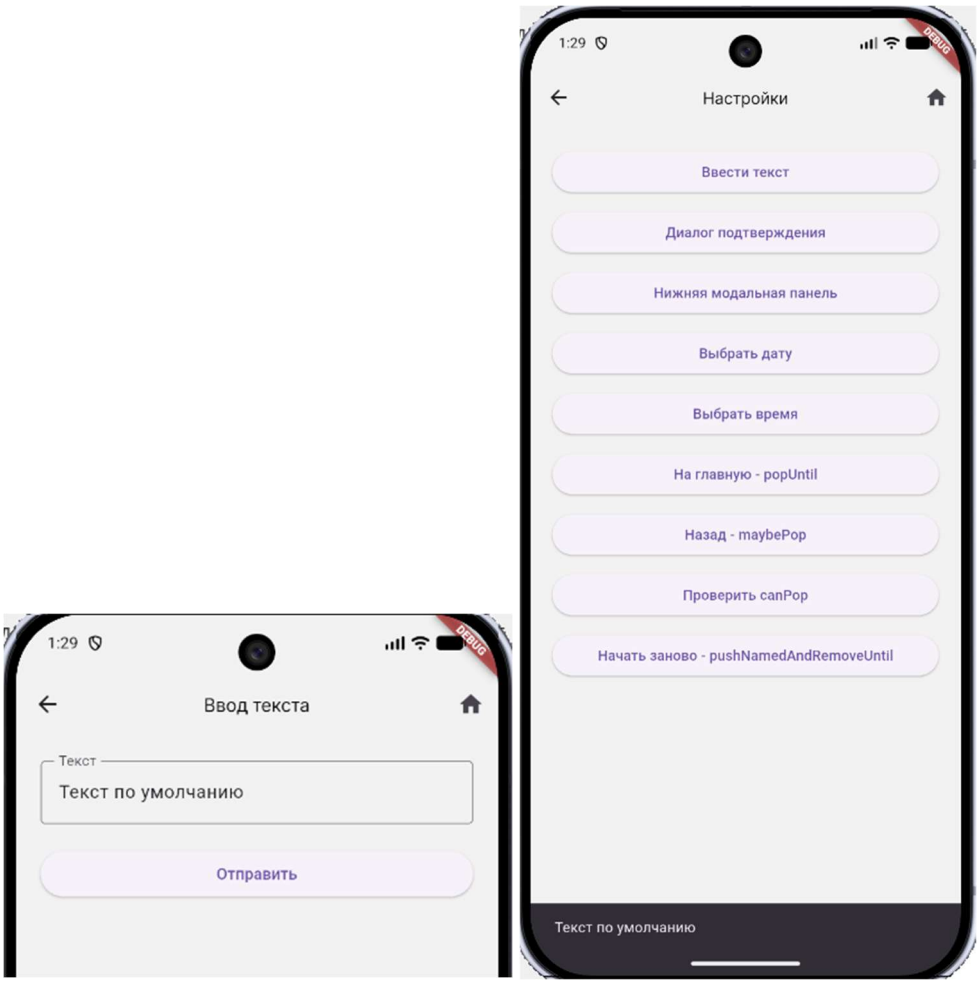


Рисунок 4 – экран ввода текста и экран настроек

4. «Для одного из экранов реализовать передачу данных в обоих направлениях»

При открытии SendTextPage передается исходный текст через arguments и отображается в поле ввода. Пользователь может изменить этот текст и вернуть его обратно на экран настроек.

```
Future<void> _sendText(BuildContext context) async {
  final result = await Navigator.pushNamed(
    context,
    AppRoutesName.sendText,
    arguments: 'Текст по умолчанию',
  );
  if (!context.mounted || result is! String) return;
  _showResult(context, result);
}
```

Рисунок 5 – передача текста из экрана настроек на экран отправки текста

5. «Продемонстрировать работу с методами pushNamedAndRemoveUntil, popUntil, maybePop, canPop»

В настройках добавлена кнопка с вызовом pushNamedAndRemoveUntil(). Она открывает новый главный экран и удаляет все предыдущие маршруты из стека.

Добавлен пример возврата на главный экран с помощью Navigator.popUntil() и проверки имени маршрута. Этот способ закрывает промежуточные экраны, сохраняя существующий главный экран.

Добавлена кнопку maybePop(), которая пытается выполнить возврат с учётом правил текущего маршрута. Также добавлена кнопку canPop(), которая проверяет наличие возможности возврата и показывает результат в SnackBar.

6. «Реализовать вызов модальных элементов: dialog, modalBottomSheet, datePicker, timePicker и уметь обрабатывать их результаты»

Диалог Реализовали через showDialog() с вариантами ответа «Да» и «Нет». Выбранный ответ возвращается вызывающему экрану и отображается в SnackBar.
Добавлен элемент showModalBottomSheet() с выбором периода отображения курсов. После выбора панель закрывается, а полученный результат показывается пользователю.

Добавлен календарь через showDatePicker() и окно выбора времени через showTimePicker(). Результаты форматируются и отображаются в SnackBar, а отмена выбора обрабатывается без сообщения.

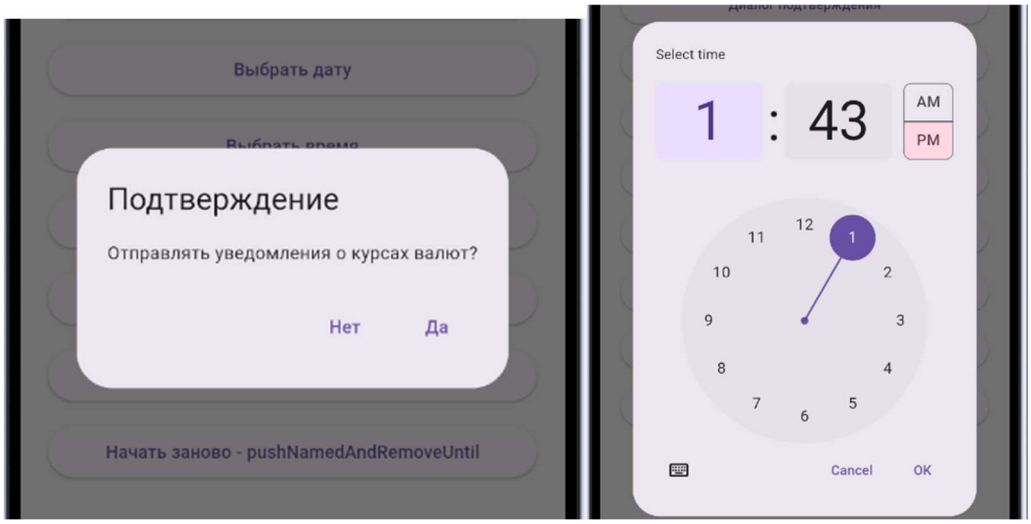


Рисунок 6 – элементы диалога и выбора времени

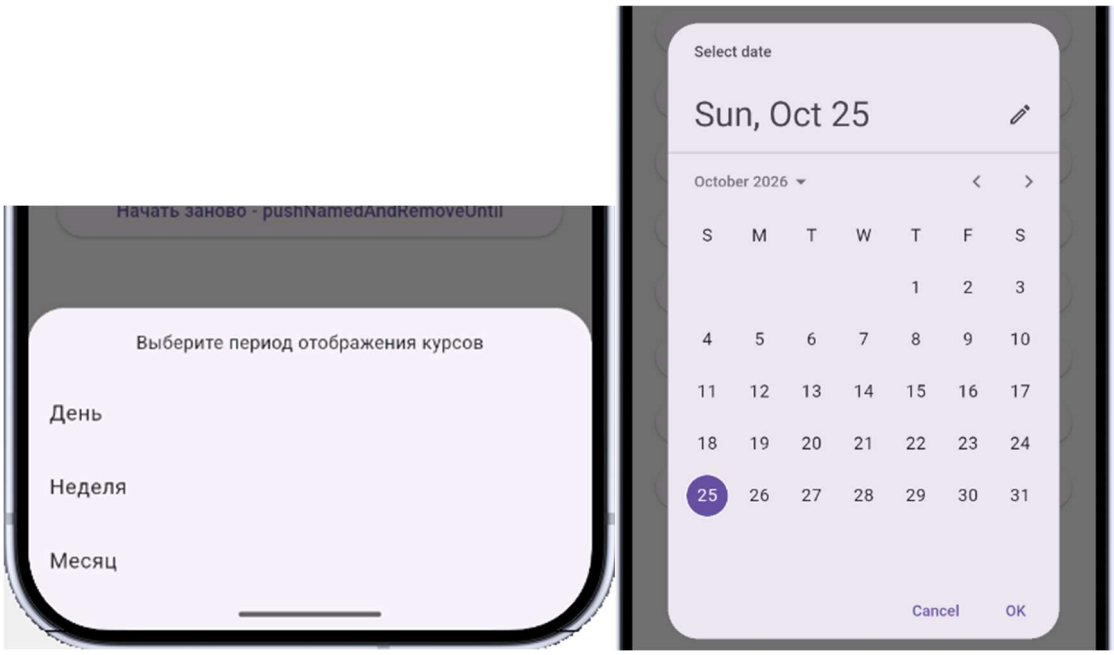


Рисунок 7 – элементы modalBottomSheet и выбора даты

7. «Реализовать кастомный анимационный переход между двумя экранами»

Маршрут экрана ввода текста перенесен в onGenerateRoute и создан через PageRouteBuilder. С помощью FadeTransition реализовано плавное появление при открытии и исчезновение при закрытии экрана.

```
onGenerateRoute: (settings) {
  if (settings.name != AppRoutesName.sendText) return null;
  final text = settings.arguments as String? ?? '';
  return PageRouteBuilder<String>(
    settings: settings,
    transitionDuration: const Duration(milliseconds: 500),
    reverseTransitionDuration: const Duration(milliseconds: 500),
    pageBuilder: (_, animation, secondaryAnimation) =>
        SendTextPage(initialText: text),
    transitionsBuilder: (_, animation, secondaryAnimation, child) {
      return FadeTransition(opacity: animation, child: child);
    },
  ); // PageRouteBuilder
},
```

Рисунок 8 – реализация перехода между экранами

8. «Добавить возможность возврата к корневому экрану из любого места в приложении (реализовать popToRoot).»

На вложенные экраны добавена иконка, вызывающая popUntil((route) => route.isFirst). Это реализует popToRoot для общего стека приложения.

```
IconButton(
  tooltip: 'На главный экран',
  icon: const Icon(Icons.home),
  onPressed: () {
    Navigator.of(context).popUntil((route) => route.isFirst);
  },
), // IconButton
```

Рисунок 9 – реализация popToRoot